Windows Flash Tool

Peter Vranken

Table of Contents

1. Connection test with DEVKIT-MPC5748G . . . 1
2. Command line switches of the flash tool . . . 3
3. Connecting to the FBL . . . 7
   3.1. Flash tool and target board are the only CAN nodes . . . 8
   3.2. Flash tool and target board are connected to a CAN bus . . . 8
4. Typical modifications of the flash tool . . . 9
   4.1. Add another MCU's memory map . . . 9
5. Building and deploying the flash tool as standalone application . . . 10
   5.1. Building the flash tool . . . 10
   5.2. Deploying the flash tool . . . 10
   5.3. Compatible Java revisions . . . 11
6. License . . . 11
   6.1. Use as a Java application or library . . . 11
   6.2. JNI and native components . . . 11
   6.3. No license on firmware or hardware . . . 11

This repository provides a CAN-based flash tool for embedded devices. The tool uses CCP as the communication protocol with a flash bootloader (FBL) running on the embedded target. CCP commands are used to erase the flash ROM and to download new firmware.

The tool has been developed and tested on Windows. It supports the inexpensive PEAK PCAN-USB adapter as CAN hardware. The implementation is written in Java, and since PEAK drivers and JNI libraries are available for Linux, the flash tool should in principle work there as well, although this has not yet been tested.

A related repository contains an FBL for the NXP MPC5748G MCU, which works with this flash tool out of the box. Other FBL implementations will likely require adjustments either in the FBL or in this tool to ensure mutual compatibility.

1. Connection test with DEVKIT-MPC5748G

A basic connection test can be performed using the NXP DEVKIT-MPC5748G board. A compatible flash bootloader is available. Make sure the FBL has been flashed onto the board beforehand.

Connect a PEAK PCAN-USB adapter to a USB port of your Windows PC. Then connect the CAN plug of the PCAN-USB to plug P5 on the DEVKIT-MPC5748G (taking termination into account).

Open a PowerShell window and verify that Java JDK is available:

```
java -version
javac -version
```

The version should be Java 18 or newer. Older versions may work but have not been tested.

Build the flash tool:

```
cd <repoCloneDirectory>
.\gradlew installDist
```

Display the flash tool's help text and then enumerate the available, connected PEAK PCAN-USB adapters:

```
.\winFlashTool.ps1 --help
.\winFlashTool.ps1 --enumerate-CAN-devices
```

The following steps require communication with the FBL. They will work only if the FBL was built with its default configuration (as provided in the GitHub repository):

- CAN Classic with 500 kBd.
- The default authentication key pair is used.

- The CAN IDs for CCP CRO and DTO messages are 0x600 and 0x650, respectively.
- The CCP station address is 0x1234, or 4660.
- DEVKIT-MPC5748G and the PCAN-USB adapter are the only connected CAN nodes.

Ensure that the DEVKIT-MPC5748G is powered off or held in reset (press and hold the push button `RESET`). Now start the flash tool so it waits for the FBL to start after a delay:

```
.\winFlashTool.ps1 --station-address 4660 --ignore-ack-err-for-connect --key-file
.\privateKey.sk --upload-version-fbl
```

Note, the same command can be abbreviated by using the short form of the command line switches:

```
.\winFlashTool.ps1 -a 4660 -ie -sk .\privateKey.sk -uv
```

Shortly after running one of these commands, start the DEVKIT-MPC5748G. Release the reset button or connect the USB cable. You have about 20 seconds to start the FBL.

The flash tool should then print the FBL version information in its console output. You should get something like:

```
winFlashTool 0.9.0.77 Copyright (C) 2025-2026, Peter Vranken
(mailto:Peter_Vranken@Yahoo.de)
    439 INFO  winFlashTool.can.CanDevice: PEAK PCAN-USB device PCAN_USBBUS2 has been
automatically chosen for operation.
    512 INFO  winFlashTool.digitalSignature.DigitalSignature: Authentication key
successfully read from <somePath>\privateKey.sk.
   5342 INFO  winFlashTool.ccp.CcpCommandConnect: ECU is connected.
   5350 INFO  winFlashTool.ccp.CcpCommandDiagService: Diagnostic service 0x00 responds
with 8 Byte of data.
   5350 INFO  winFlashTool.ccp.CcpCommandUpload: Upload 0x8 Byte from target.
   5355 INFO  winFlashTool.ccp.CcpCommandUpload: 0x000008 Byte uploaded (100%).
   5395 INFO  winFlashTool.ccp.CcpCommandsDownloadProgram: Download 0x40 Byte to
memory address 0x40001048.
   6082 INFO  winFlashTool.ccp.CcpCommandsDownloadProgram: 0x000040 Byte downloaded
(100%).
   6085 INFO  winFlashTool.ccp.CcpCommandDiagService: Diagnostic service 0x01 responds
with 125 Byte of data.
   6085 INFO  winFlashTool.ccp.CcpCommandUpload: Upload 0x7D Byte from target.
   6127 INFO  winFlashTool.ccp.CcpCommandUpload: 0x00007D Byte uploaded (100%).
   6130 INFO  winFlashTool.ccp.CcpCommandDisconnect: ECU is disconnected.
   6130 INFO  winFlashTool.WinFlashTool: Version flash bootloader:
DEVKIT-MPC5748G - Flash Bootloader
Copyright (C) 2017-2026  Peter Vranken
Version 0.7.0 (Configuration: DEBUG in flash ROM)
   6275 INFO  winFlashTool.WinFlashTool: winFlashTool terminating with 0 errors and 0
```

```
warnings.
```

If the expected output does not appear, you have two initial debugging options:

1. Add `--log-level DEBUG` or `--log-level TRACE` to the flash tool command line and repeat the procedure. This provides more detailed diagnostics.
2. Open a terminal application, connect to the COM port of the DEVKIT-MPC5748G, and repeat the procedure. See its readme for instructions. This reveals how the FBL reacts and responds.

> If you have just flashed the FBL onto the DEVKIT-MPC5748G but have not yet flashed an application, then neither the reset button nor delayed board power-up is required. The FBL starts immediately after reset and remains active, listening for a CAN CCP connection at all times.

If this connection test succeeds, then downloading an application, uploading memory, or erasing flash should work without difficulty.

As a last example: Download and flash the sample application from the FBL repository, upload it again for verification and eventually reset the board in order to start the flashed application:

```
$fblDir = "<cloneDirOfFblRepository>"
$app = "$fblDir\applications\startup\bin\ppc\PRODUCTION\DEVKIT-MPC5748G-startup.s28"
.\winFlashTool.ps1 -m MPC5748G -a 4660 -sk privateKey.sk -ie -i $app -r APP
```

2. Command line switches of the flash tool

Common syntax rules:

- A command line switch and its value or argument must always be separated by a blank. A form such as `--log-level=OFF` is not recognized.
- Switches that do not require a value or argument must not be combined. A construct like `-de` is not interpreted as `-d -e`.

Table 1. Command line switches of flash tool

Command	Abbr.	Meaning
--help	*-h*	*Demand this help.*

Command	Abbr.	Meaning
--use-standard-log4j2 -configuration	*-V*	*Use the standard Log4j2 configuration sequence. See http://logging.apache.org/log4j/2.x/manual/configuration.html for details. If this option is used, the programmatic configuration is disabled, and the other logging-related switches (`--log-level`, `--log-file`, `--log4j2-pattern`) are ignored.* *Optional; by default, programmatic logging configuration is used.*
--log-level	*-v*	*Sets the verbosity of all logging. Allowed values are OFF, FATAL, ERROR, WARN, INFO, and DEBUG.* *Optional; default is INFO.*
--log-file	*-l*	*If specified, a log file will be written containing general program flow messages.* *Optional; by default, logging output is written only to the console.*
--log4j2-pattern	*-p*	*Specifies the pattern used for log file entries, for example, "%d %C %p: %m%n". See http://logging.apache.org/log4j/2.x/manual/layouts.html#PatternLayout for details.* *Optional; the default pattern is usually sufficient.* *Please note, the console output is not affected.*
--mcu-target	*-m*	*The MCU to be programmed. Supported targets are: `MPC5748G`, `MPC5775B`, `MPC5775E`, and `MPC5777C`.* *This argument is mandatory for regular operation but is not required when using `--enumerate-CAN-devices` or `--generate-key-pair`.*
--CAN-device	*-d*	*Selects the CAN device to use. Consider running `--enumerate-CAN-devices` to list connected and available devices.* *Optional; by default the first available device is used.*
--CAN-configuration	*-cfg*	*CAN device configuration. For classic CAN, this is a single integer specifying the Baud rate in kBd. For CAN FD, this is the configuration string defined by the PCAN Basic API. See the PCAN documentation for details.* *Optional; default is classic CAN at 500 kBd.*

Command	Abbr.	Meaning
--CAN-ID-CRO	*-tx*	*The CAN ID used for all CCP CRO Tx messages. Extended CAN IDs are not yet supported.* *Optional; default is the 11-bit ID 0x600.*
--CAN-ID-DTO	*-rx*	*The CAN ID used for all CCP DTO Rx messages. Extended CAN IDs are not yet supported.* *Optional; default is the 11-bit ID 0x650.*
--ignore-ack-err-for-connect	*-ie*	*Ignores CAN Rx errors when sending the CCP CONNECT command and waiting for its response, which can simplify connecting to an FBL that checks for connection requests only shortly after reset.* *When this option is used, retrying the CONNECT command becomes counter-productive. Instead, a larger timeout is required. For this purpose, the switch* `--no-retries-ccp-connect` *is repurposed to specify the timeout: it becomes (retries + 1) seconds.* *Optional; by default, CAN errors abort the connection attempt.*
--no-retries-ccp-connect	*-nr*	*Number of retries (after short delay) if the initial CCP connect fails. Permitted range is [0, 1000].* *Optional; default is 20.*
--station-address	*-a*	*The 16-bit station address of the CCP target.* *Optional; default is station address 0.*
--key-file	*-sk*	*File containing the private key for the Ed25519 authentication procedure with the FBL.* *Optional; default is to skip authentication.*
--upload-version-fbl	*-uv*	*Uploads the version information of the FBL in the target ECU and prints it in the log.* *This option is target-specific and assumes the target provides the FBL version information as result of CCP command DIAG_SERVICE with service number 0.* *Optional; disabled by default.*

Command	Abbr.	Meaning
--srec-output-file	*-o*	*Specifies the S-record file to write uploaded memory contents to. If used, an upload from the target is performed. If combined with erase or program commands, the upload happens first, prior to modifying the flash ROM contents.* *Optional.*
--address-range	*-u*	*Address range for upload, given as two hexadecimal addresses separated by a colon, e.g., 00800000:008E0000. May be used multiple times.* *This option is mandatory when using* `--srec-output-file`*.*
--srec-input-file	*-i*	*S-record file with memory contents to be programmed or verified. If used without* `--verify-only`*, the file is downloaded to the target and programmed.* *Optional.*
--verify-only	*-vo*	*Modifier for* `--srec-input-file`*. Loads the S-record file and compares it to the memory contents on the target without modifying any memory.* *Optional; default is to program the file.*
--no-verify	*-nv*	*Modifier for* `--srec-input-file`*. Flash programming is normally followed by an upload and comparison of the programmed memory contents with the input file. However, this roughly doubles the duration of the complete programming procedure. Using option* `--no-verify` *means to skip the upload and data comparison to save the additional time.* *Optional, default is doing the verification.* *Please note, regardless of this switch, an immediate verify of each flash programming step is always performed. However, this validates only the physical flash programming but not potential data transmission errors.*
--erase-all	*-e*	*Erases all flash ROM controlled by the FBL. May be used with or without download and programming.* *If not used while* `--srec-input-file` *is used to command the download, then only the flash blocks needed for the S-record contents are erased.*

Command	Abbr.	Meaning
--reset-target-ECU	*-r*	*Requests a reset of the ECU at the end of communication. The argument must be either "APP" or "FBL", telling the FBL whether to start the flashed application or restart itself after reset.* *Please note, this command depends on the FBL in the target ECU. The flash tool just submits the CCP command DIAG_SERVICE with service number 8 (APP) or 2 (FBL). It depends on the FBL implementation, how it reacts on these commands.* *Optional; default is no reset. The FBL remains alive in the target and the flash tool could re-connect.*
--dry-run	*-n*	*Runs the flash tool without modifying flash contents. All potentially harmful CCP commands are skipped, but CONNECT and DISCONNECT are still executed. Useful for validating the setup without risk.* *Optional; default is normal operation.*
--enumerate-CAN-devices	*-dev*	*Lists connected PEAK PCAN-USB devices and exits. If* `--CAN-device` *is also given, identification mode is enabled: the selected device blinks its LED in orange for a short period.* *Useful for a check of the setup.* *Optional, default is not to list devices.*
--generate-key-pair	*-kp*	*If given, then the flash tool will generate an ED25519 key pair, which can be used for the authentication procedure with the FBL. The public key is printed to the log and would be integrated in the FBL.* *This switch has a file designation as argument. The private key is written into that file and would be used with* `--key-file`*, when later using the flash tool to communicate with the FBL.* *The tool exits after generating the key pair.* *Optional, default is not to generate a key pair.*

3. Connecting to the FBL

There are several ways to establish the connection between the flash tool and the FBL. Some of

these are application-specific; the flashed application has one or more trigger options to switch from the running application to the FBL, which is then available for CCP communication. These solutions are application-specific and will not be discussed further here.

Furthermore, many FBLs share the characteristic that, regardless of these specific solutions, they allow contact with the flash tool immediately after each reset. The MCU reset starts the FBL, which then waits a short time for a possible connection via CAN and CCP. Only if this time has elapsed without a connection being established does the FBL start the flashed application. This procedure is a backdoor to allow reprogramming of the application even if the already flashed application has an error and the normally used, specific solution therefore fails.

Unconditionally waiting for a possible connection means an undesirable increase in the application's startup time. This time is therefore kept to a minimum. This, in turn, makes it difficult to start the flash tool at precisely the right point. Our flash tool offers two options to simplify the process.

3.1. Flash tool and target board are the only CAN nodes

If the flash tool is connected to the board running the FBL via a direct point-to-point connection, meaning there are no other CAN communication participants besides these two, the `--ignore-ack-err-for-connect` option can be used.

With this option, the flash tool is started while the board with the FBL is still inactive. (It may be helt in reset or still switched off.) The flash tool sends the CCP `CONNECT` message. This message remains unheard on the CAN bus because there is (yet) no other active node that would send the acknowledgment. In this situation, CAN transceivers (such as those used in the PEAK PCAN-USB) continuously resend the message until an acknowledgment is finally received. At the same time, the situation is reported back as a CAN protocol error. The flash tool sees the error but ignores it due to the `--ignore-ack-err-for-connect` option. Normally, such an error would abort the flashing process, but now the tool simply continues waiting for a response from the FBL.

The board is switched on with a slight delay. The FBL starts, initializes the CAN hardware, and then receives the next repetition of the CCP `CONNECT` message. Simultaneously, the CAN hardware sends the acknowledgement. The FBL evaluates the message and sends the response — the connection is established.

The maximum time the flash tool waits for the `CONNECT` response is configurable; the default value is 20 seconds. The board with the FBL must be powered on within this timeframe after the flash tool starts.

3.2. Flash tool and target board are connected to a CAN bus

If the flash tool and board are connected to a CAN bus that also contains other CAN devices, `--ignore-ack-err-for-connect` will not help. Again, the idea is to start the flash tool first and then, after a short delay, the board with the FBL. The situation is now significantly different; the other

CAN nodes would send the acknowledgement as soon as the CCP `CONNECT` message is sent for the first time. The flash tool would assume that the `CONNECT` was understood and would be answered shortly afterward — which doesn't happen because the board with the FBL is still powered off. The flash tool would then abort the flashing process with an error message.

In this situation, the other option `--no-retries-ccp-connect` helps. If a CCP `CONNECT` is not answered promptly, the flash tool sends a repeated `CONNECT` after a short wait, as many times as the option specifies. If the board with the FBL is powered on after a short delay, the next of these repeated send attempts should fall within the small time window that the FBL maintains for establishing contact. The FBL responds normally, and the connection is established.

The flash tool repeats sending the CCP `CONNECT` messages approximately every 30 ms. Therefore, the time window in the FBL should be configured to approximately 50 ms (or more) to guarantee that at least one flash tool send repetition is seen.

Please note that the flash tool's 30 ms repetition time is a compile-time constant and cannot be changed via command-line arguments.

The default value for `--no-retries-ccp-connect` is 20. If you can reliably start flash tool and board within half a second, this option is not needed.

4. Typical modifications of the flash tool

It is unlikely that you can use the flash tool without any modification. The tool works out of the box only if:

- You use an MPC5748G board together with the FBL from the related repository.
- You migrate the FBL from the related repository to one of the similar MCUs MPC5775B/E or MPC5777C and use it.
- You have your own FBL implementation that uses CCP and supports the same CCP commands in the same manner.

In all other situations, some level of adaptation will be required. The smallest necessary modification is typically adding support for another MCU type.

4.1. Add another MCU's memory map

The flash tool is aware of the target MCU only to the extent needed to optimize CCP erase operations. Erase commands are derived from the flash block layout of the MCU. S-records in the input file refer to specific flash regions, and only those regions containing at least one byte to be programmed are erased. Knowledge of the flash block structure is also used to validate the S-record input: any S-record referencing an address outside the supported flash range results in an early error, even before any communication with the ECU begins.

To add support for a new MCU, create a memory map class derived from `winFlashTool.mcu.Flash`. Use `Mpc5748G_C55FMC.java` as a template; you mainly need to fill in a table describing the addresses of the flash blocks managed by your FBL.

To make the new MCU selectable, extend the command-line handling in `WinFlashTool.java`:

1. In method `defineArguments`, update the help text for the `--mcu-target` option so your new MCU appears in the list of supported devices.
2. In method `run` (around line 675), a switch/case statement creates the appropriate `Flash` object based on the selected MCU. Add a case for your new MCU and instantiate your new memory map class.

5. Building and deploying the flash tool as standalone application

5.1. Building the flash tool

The build of the Java application is under control of Gradle. After cloning the repository, you'd open a shell window (e.g. PowerShell) and `cd` into the root of the chosen clone directory:

```
cd <cloneDirOfFlashToolRepository>
gradlew installDist
```

The very first build may take several minutes. Gradle will download all if its own libraries from the Internet and all libraries required for the flash tool. (Subsequent builds are much faster as this doesn't need to be repeated.)

The Gradle task `installDist` yields the folder `build\install\winFlashTool`. This folder is the ready to use bundle of jar files and other libraries plus the inevitable Java launch script. If you followed the instruction given in Section 1 and ran the flash tool using `winFlashTool.ps1` then you effectively ran the Gradle created Java launch script `build\install\winFlashTool\bin\winFlashTool.bat`; the PowerShell script is a one-liner for calling the script generated by Gradle.

5.2. Deploying the flash tool

The Gradle build has created the ready-to-use bundle of files, directly usable as folder `build\install\winFlashTool`. You can copy this folder anywhere, e.g., to `C:\Program Files\`.

Alternatively, you can run this Gradle command:

```
gradlew distZip
```

It creates the same bundle of needed files as ZIP archive `build\distributions\winFlashTool-<version>.zip`. You can distribute it and extract the contents at any location, e.g., `C:\Program Files\`.

Using the flash tool means running the launch script in folder `<chosenTargetDir>\winFlashTool\bin` (or `<chosenTargetDir>\winFlashTool-<version>\bin` if you extracted the ZIP archive). To make this script visible and accessible from any working directory, you should add this folder to the Windows

search PATH. To check the installation, open a new shell window (e.g., PowerShell) and type:

```
where.exe winFlashTool
```

The response should point to your chosen installation folder. Now try:

```
winFlashTool --help
```

5.3. Compatible Java revisions

The flash tool has been successfully built and tested with OpenSDK Java 23.0.1 and 18.0.1.1.

6. License

This project is licensed under the **GNU Lesser General Public License v3.0 or later** (`LGPL-3.0-or-later`).

6.1. Use as a Java application or library

You may:

- Use this software as a standalone application.
- Use the Java components as a library in proprietary or open-source projects.
- Link against the Java classes dynamically at runtime.

Provided that:

- Modifications to the LGPL-licensed components themselves are made available under the same license.

6.2. JNI and native components

This project contains native components accessed via **JNI**.

- The JNI boundary is considered a *dynamic linking interface*.
- Applications may link to the native libraries without being subject to LGPL licensing, as long as the LGPL-covered code itself is not modified.
- If you modify the native LGPL-covered code, you must make the modified source available under the LGPL.

6.3. No license on firmware or hardware

This license does **not** apply to:

- External hardware
- Target firmware
- Vendor-supplied binaries or drivers